

Normalization Techniques & Attention Mechanisms

CS290W LLM06 Lecture

LU Yuzhou

Lecture Overview

This lecture covers two foundational building blocks of Transformers:

Part 1 — Normalization Techniques

- Why normalization is necessary for deep networks
- LayerNorm vs RMSNorm
- Activation functions: ReLU, GELU, SiLU
- Feed-Forward Networks (FFN)
- Pre-Norm vs Post-Norm architectures

Part 2 — Attention Mechanisms

- Self-attention: Query, Key, Value
- Scaled dot-product attention
- Causal masking for autoregressive models
- Multi-head attention

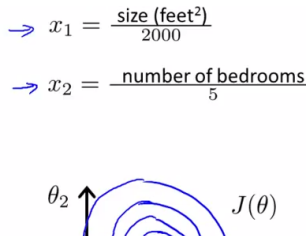
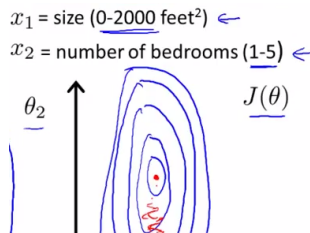
Why do we need normalization?

Why Normalization Matters

Deep networks suffer from **internal covariate shift**: the distribution of each layer's inputs changes during training.

Problems Without Normalization

- **Activation scale variations**: values range from -192 to $+216$ in a small network
- **Gradient flow problems**: vanishing or exploding gradients
- **Training instability**: requires very small learning rates
- **Slow convergence**: takes many epochs to reach good performance



Layer Normalization

LayerNorm normalizes across the feature dimension for each token position:

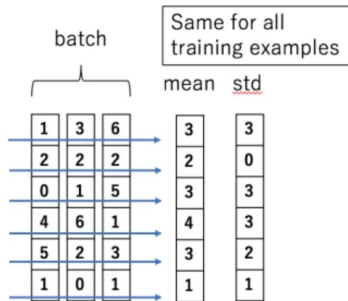
$$\text{LayerNorm}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \cdot \gamma + \beta$$

$$\mathbf{h} = \mathbf{g} \odot N(\mathbf{x}) + \mathbf{b}, \quad N(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sigma}, \quad \mu = \frac{1}{H} \sum_{i=1}^H x_i, \quad \sigma = \sqrt{\frac{1}{H} \sum_{i=1}^H (x_i - \mu)^2}$$

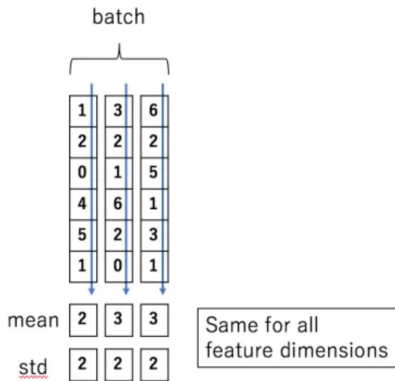
- γ (gain): learnable scaling, initialized to ones
- β (bias): learnable shifting, initialized to zeros
- Output has **zero mean** and **unit variance**

Layer Normalization: Diagram

Batch Normalization



Layer Normalization



Key Property

RMS Normalization (RMSNorm)

RMSNorm is a simplified alternative to LayerNorm, used in **LLaMA** and other modern models.

$$\text{RMSNorm}(x) = \frac{x}{\text{RMS}(x)} \cdot \gamma, \quad \text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}$$

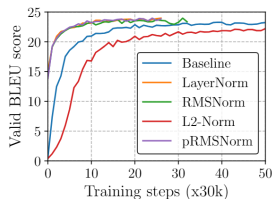


Figure 2: SacreBLEU score on newstest2013 for the RNNSearch. Models are implemented according to *Nematus* [25] in Tensorflow.

Model	Test14	Test17	Time
Baseline	21.7	23.4	399±3.40s
LayerNorm	22.6	23.6	665±32.5s
L2-Norm	20.7	22.0	482±19.7s
RMSNorm	22.4	23.7	501±11.8s (24.7%)
pRMSNorm	22.6	23.1	493±10.7s (25.9%)

Table 2: SacreBLEU score on newstest2014 (Test14) and newstest2017 (Test17) for RNNSearch using Tensorflow-version Nematus. “Time”: the time in second per 1k training steps. We set p to 6.25%. We highlight the best results in bold, and show the speedup of RMSNorm against LayerNorm in bracket.

RMSNorm: Implementation

```
class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-6):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim))

    def _norm(self, x):
        # RMS = sqrt(mean(x^2) + eps)
        rms = x.pow(2).mean(-1, keepdim=True).add(self.eps).sqrt()
        return x / rms

    def forward(self, x):
        output = self._norm(x.float()).type_as(x)
        return output * self.weight
```

- Cast to float32 for numerical stability, then cast back
- Only 1 learnable parameter per feature dimension (vs 2 for LayerNorm)

LayerNorm vs RMSNorm: Comparison

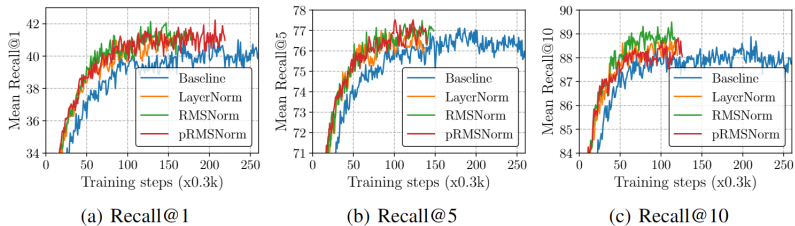
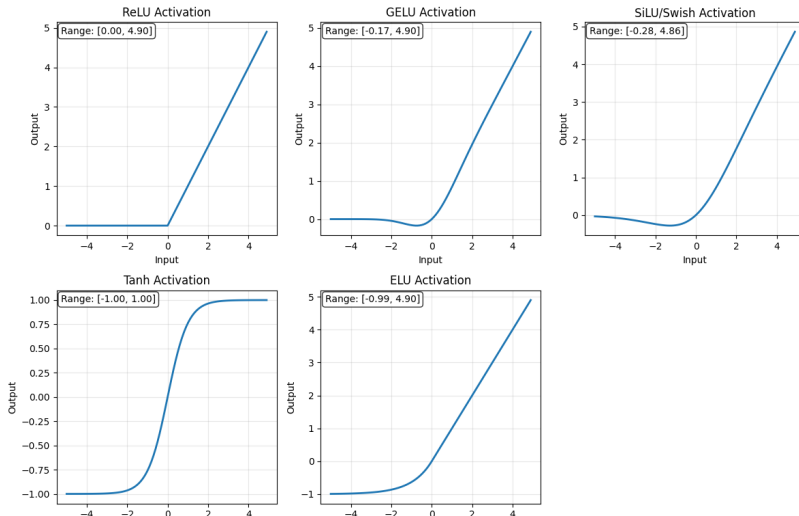


Figure 6: Recall@K values on validation set for the order-embedding models.

Property	LayerNorm	RMSNorm
Mean subtraction	Yes	No
Variance normalization	Yes	Yes (RMS)
Learnable parameters	γ, β	γ only
Output mean	≈ 0	Preserved
GPU speed	Baseline	$1.1\text{--}1.3\times$ faster
Used in	BERT, GPT-2	LLaMA, Mistral

Activation Functions in Modern LLMs

Activation functions introduce **non-linearity**, enabling models to learn complex patterns.



Feed-Forward Network (FFN) in Transformers

Each Transformer block contains an FFN after the attention layer:

$$\text{FFN}(x) = W_2 \cdot \text{activation}(W_1 x + b_1) + b_2$$

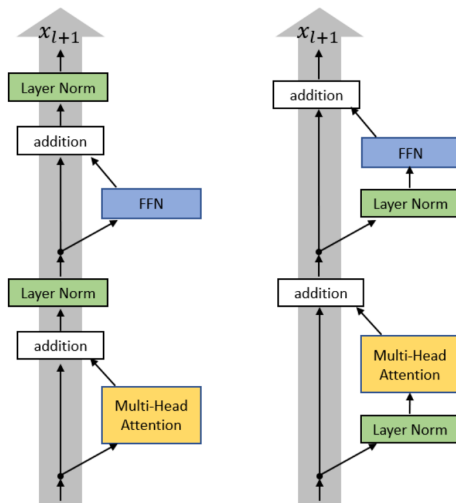
- W_1 : up-projection, $d_{\text{model}} \rightarrow d_{\text{ff}}$ (typically $4\times$ expansion)
- W_2 : down-projection, $d_{\text{ff}} \rightarrow d_{\text{model}}$
- Activation: GELU (BERT, GPT-2) or SiLU (LLaMA)

Design Choices

- Expansion ratio: typically $4\times$ (e.g., $768 \rightarrow 3072$ in BERT)
- FFN contains **most parameters** in a Transformer block
- Dropout for regularization (typically $p = 0.1$)
- Residual connections prevent vanishing gradients

Pre-Norm vs Post-Norm Architectures

The **placement** of normalization significantly impacts training dynamics.



Pre-Norm vs Post-Norm: Detailed Comparison

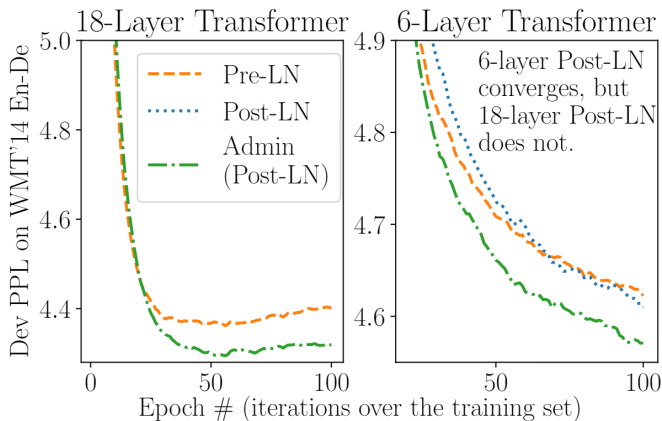


Figure 1: Lacking enough robustness and stability, the 18-Layer Post-LN Transformer training (*i.e.* the original architecture) diverges and is omitted in the left graph

Pre-Norm vs Post-Norm: Code

Post-Norm:

```
def forward(self, x):
    # Attention: apply, add residual, normalize
    x = self.ln1(x + self.attention(x))
    # FFN: apply, add residual, normalize
    x = self.ln2(x + self.ffn(x))
    return x
```

Pre-Norm:

```
def forward(self, x):
    # Attention: normalize, apply, add residual
    x = x + self.attention(self.ln1(x))
    # FFN: normalize, apply, add residual
    x = x + self.ffn(self.ln2(x))
    return x
```

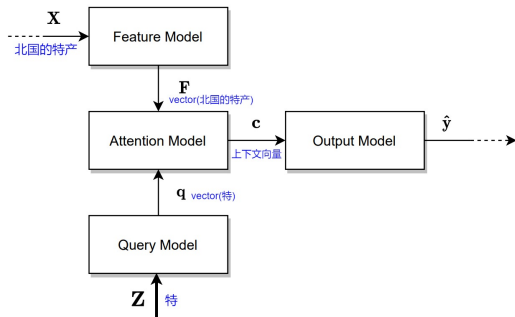
Key Insight

Pre-Norm keeps a **clean residual path** from input to output, improving gradient flow in deep networks.

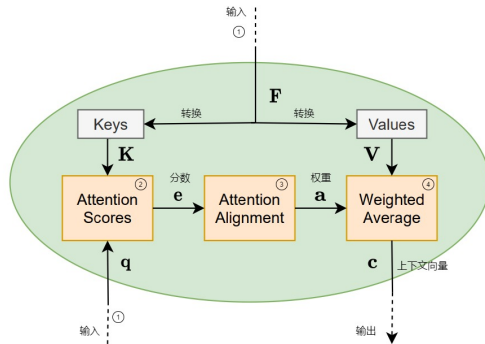
The Heart of Transformers

Self-Attention: The Big Idea

Self-attention allows each position in a sequence to **attend to all other positions**.



原始图片来源: 论文 A General Survey on Attention Mechanisms in Deep Learning

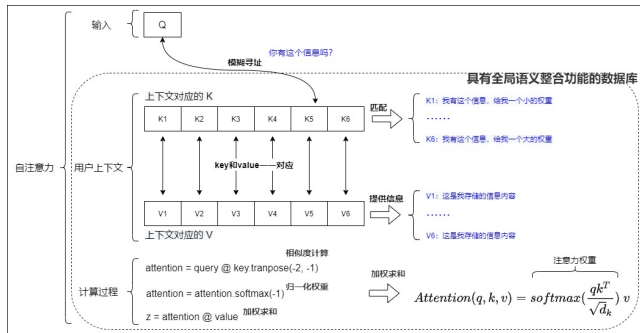


原始图片来源: 论文 A General Survey on Attention Mechanisms in Deep Learning

Three roles per token:

- **Query (Q):** "What am I looking for?" — what information does this position need?
- **Key (K):** "What do I have to offer?" — what information does this position contain?

Scaled Dot-Product Attention



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V$$

Step-by-step:

- 1 **Compute scores:** $S = QK^T$ — dot product measures similarity
- 2 **Scale:** $S/\sqrt{d_k}$ — prevent extremely large values

Attention Score Intuition

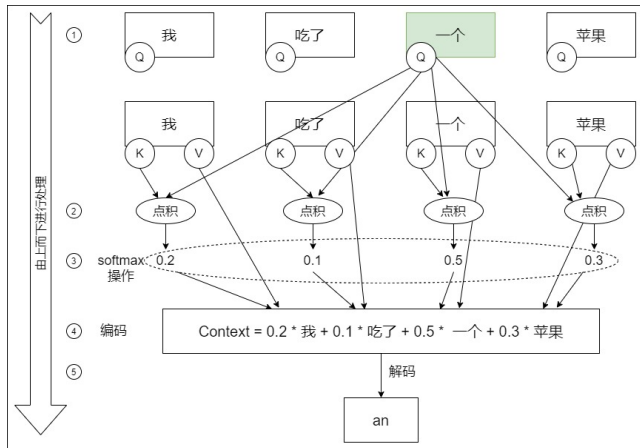
Consider the sentence: “*The cat sat on the mat because it was tired*”

- When processing “*it*”, the model needs to know *what* “it” refers to
- The Query of “*it*” asks: “Who is the subject?”
- The Key of “*cat*” says: “I am a noun, potentially a subject”
- High $Q \cdot K$ score $\rightarrow$ “*it*” attends strongly to “*cat*”
- The Value of “*cat*” flows into “*it*” representation

Key Insight

Attention scores form a **probability distribution** over all positions.
Higher scores = more information flow between those positions.

Causal Masking for Autoregressive Models



Problem: In GPT-style models, position i must **not see future tokens** $j > i$.

Solution: Apply a causal mask before softmax:

Padding Mask

For variable-length sequences, shorter inputs are padded with special tokens.

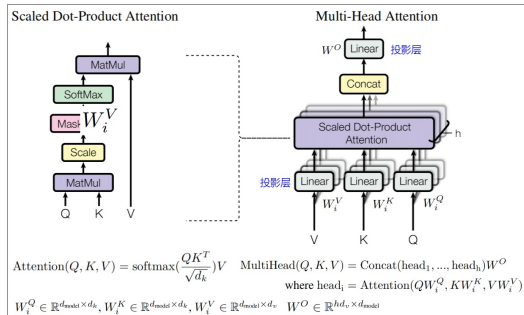
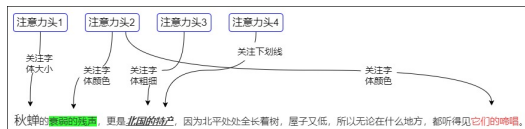
- Padding tokens are **not real content** — should not receive attention
- Set padding positions to $-\infty$ in the attention score matrix
- After softmax, padding positions get ≈ 0 weight

```
Apply mask: set padded positions to -inf scores[padding_mask.unsqueeze(1).expand(-1, seq_len, -1)] = float('-inf')
```

Multiple Masks

Causal and padding masks can be **combined** for autoregressive models with variable-length inputs.

Multi-Head Attention



$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$

- h heads (e.g., 8 or 12), $\text{head_dim} = d_{\text{model}}/h$
- Different heads learn **different relationship types** (syntax, semantics, position)
- Parallel computation $\rightarrow$ efficient on GPU

Multi-Head Attention: Reshaping

Reshape tensors to process all heads simultaneously:

```
# Q, K, V shape: [batch, seq_len, hidden_dim]
# e.g., [2, 4, 8] with num_heads=2, head_dim=4

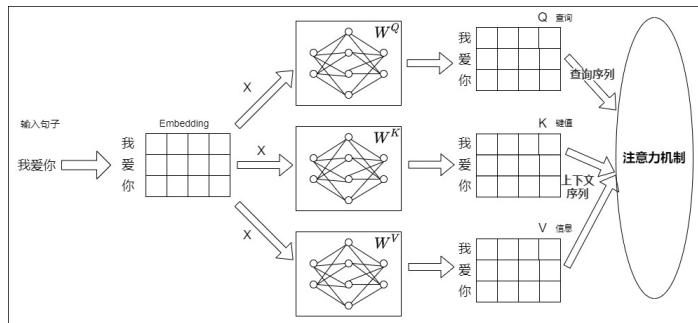
# Step 1: Reshape [batch, seq, hidden] → [batch, seq, heads, head_dim]
q_heads = q.view(batch_size, seq_len, num_heads, head_dim)

# Step 2: Transpose [batch, seq, heads, dim] → [batch, heads, seq, dim]
q_heads = q_heads.transpose(1, 2)

# Step 3: Compute attention for ALL heads simultaneously
scores = torch.matmul(q_heads, k_heads.transpose(-2, -1))
scores = scores / math.sqrt(head_dim)
weights = F.softmax(scores + causal_mask, dim=-1)
output = torch.matmul(weights, v_heads)

# Step 4: Concatenate heads back
output = output.transpose(1, 2).contiguous().view(batch, seq, hidden)
```

Visualizing Attention Patterns



- Each head produces a different attention pattern
- Some heads attend to **local context** (nearby tokens)
- Some heads attend to **long-range dependencies**
- Some heads focus on specific **syntactic patterns**

Interpretability: Attention weights show which tokens the model “focuses on” for each position.

Output Projection and Complete Pipeline

After multi-head attention, apply a final linear projection:

$$\text{output} = W_O \cdot \text{MultiHead}(Q, K, V)$$

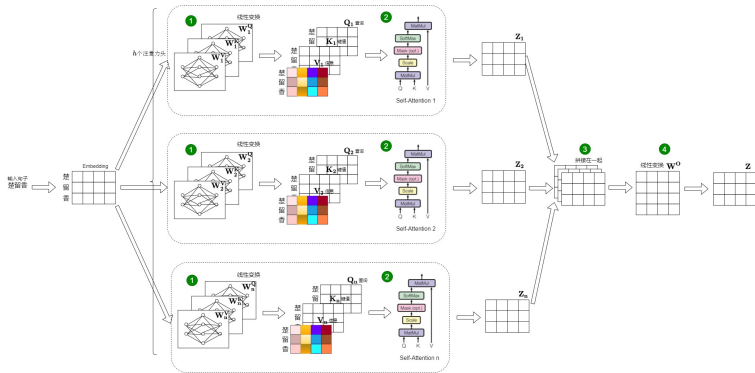
Complete attention pipeline:

- 1 Input $X \in \mathbb{R}^{B \times N \times d}$
- 2 Project to Q, K, V via W_Q, W_K, W_V
- 3 Reshape for multi-head: $[B, N, d] \rightarrow [B, h, N, d/h]$
- 4 Compute scaled dot-product attention with causal mask
- 5 Concatenate heads: $[B, h, N, d/h] \rightarrow [B, N, d]$
- 6 Final projection: W_O

Complexity

$O(N^2 d)$ — quadratic in sequence length N , linear in hidden dim d .

Summary: Key Takeaways



Normalization: LayerNorm (BERT, GPT-2) vs RMSNorm (LLaMA, Mistral); Pre-Norm preferred for deep LLMs.

Attention: Q/K/V projections $\rightarrow$ scaled dot-product $\rightarrow$ causal masking $\rightarrow$ multi-head $\rightarrow$ output projection.

- `LLM05a-Normalization-Techniques.ipynb`
 - Implement custom RMSNorm
 - Compare LayerNorm vs RMSNorm statistics
 - Analyze activation functions (ReLU, GELU, SiLU)
 - Build Transformer FFN and complete blocks
- `LLM05b-Advanced-Normalization-Applications.ipynb`
 - Implement LayerNorm from scratch
 - Pre-Norm vs Post-Norm architecture comparison
 - Gradient flow analysis
- `LLM06-Attention-Mechanisms.ipynb`
 - Implement Q, K, V projections
 - Compute attention scores with scaling and masking
 - Build multi-head attention from scratch
 - Visualize attention patterns